

0.21. Descendre avec `deplace_y`

`deplace_y(colonne, ligne, tuile, pas)` : comme `deplace_x`, mais sur l'axe Y. La lettre descend de 5 lignes.

1. Ce qu'il faut comprendre

C'est le 0.18, mais à la verticale. L'écran a deux axes : **X**, l'horizontal (les colonnes, de 0 à gauche jusqu'à 19 à droite), et **Y**, le vertical (les lignes, de 0 en haut jusqu'à 17 en bas).

Ce qui est nouveau ici : `deplace_y(colonne, ligne, tuile, pas)`. Les mêmes renseignements que `deplace_x`, dans le même ordre, mais c'est la **ligne** qui change : la lettre descend d'une ligne tous les quarts de seconde, **pas** fois. Un pas positif **descend**, car les numéros de ligne grandissent vers le bas.

Elle rend la nouvelle ligne, d'où `y = deplace_y(...)`, pour la même raison qu'au 0.18 : la fonction n'a qu'une copie de `y`.

Au bord (ligne 17), elle s'arrête au lieu de sortir de l'écran.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1595 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;      // le nombre de lignes à parcourir
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;        // la LIGNE de la lettre : elle part de 0 (tout en haut)
```

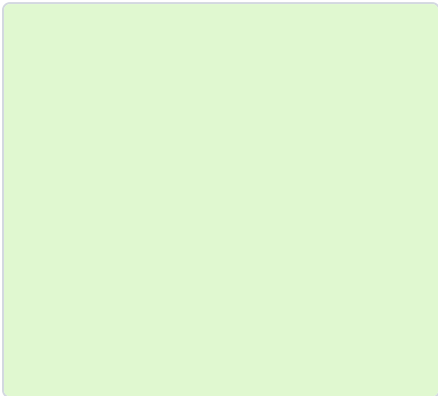
3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    // y = deplace_y(0, y, ALPHABET[0], pas);
    // | |           | | |           |
    // | |           | | |           +-- pas      : combien de lignes (5, vers le bas)
    // | |           | | +----- tuile      : la lettre A
    // | |           | +----- ligne      : d'où elle part – une COPIE de y
    // | |           +----- colonne : 0, tout à gauche (elle ne change pas)
    // | +--- deplace_y fait descendre la lettre, puis REND la ligne d'arrivée : 5
    // +----- « y = » range ce 5 dans y
    y = deplace_y(0, y, ALPHABET[0], pas);    // après cette ligne : y vaut 5

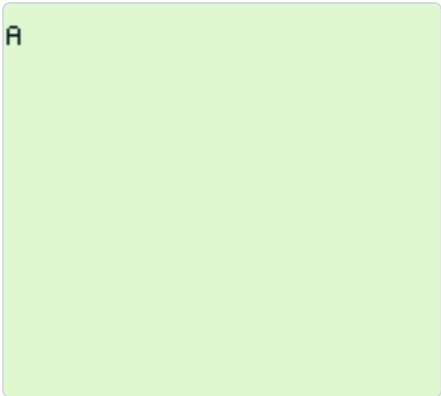
    while (true) {
        image();        // la lettre reste en ligne 5
    }
}
```

3. Ce qui se passe, étape par étape

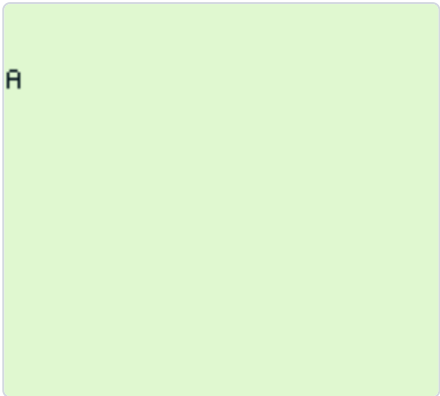
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



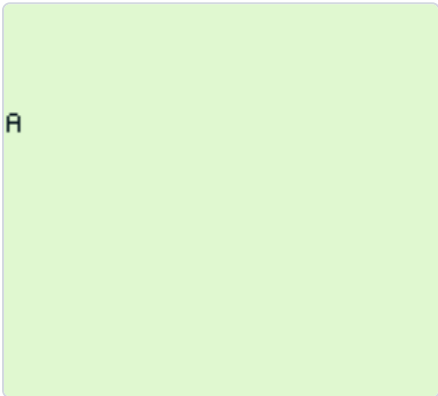
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui descend de 5 lignes, puis s'arrête en ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ y a reçu la ligne d'arrivée (y = 5)
- ✓ le A est en ligne 5, seul dans sa colonne (« A »)

Ensuite : **0.21.1. Descendre avec `deplace_y` — de base, ailleurs** — niveau 0 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++